Evaluación del módulo 7

Consigna del proyecto👨🏻‍💻

alkemy

Proyecto: *Alke Wallet*

***Módulo 7**: Desarrollo Web con Django*

Situación inicial 📍

Una empresa fintech, Alke Financial, ha solicitado el desarrollo de una billetera digital (Alke Wallet) que permita a los usuarios **gestionar y administrar sus activos financieros de forma segura y sencilla.** El equipo de desarrollo debe abordar desafíos relacionados con el acceso y gestión de datos, la creación de un modelo de datos robusto, la sincronización del esquema mediante migraciones, la ejecución de consultas personalizadas y la implementación de operaciones CRUD, utilizando las herramientas y buenas prácticas que ofrece Django. El proyecto se realizará en Visual Studio Code, utilizando librerías gratuitas y configuraciones que faciliten el desarrollo y la posterior escalabilidad del producto.

Nuestro objetivo 📋

El objetivo de "Alke Wallet" es desarrollar una **aplicación web funcional** que permita a los usuarios crear y gestionar sus cuentas digitales, realizar transacciones, consultar saldos y generar reportes. Se busca garantizar la integridad y seguridad de los datos mediante el uso del ORM de Django, el manejo de migraciones y la integración de aplicaciones preinstaladas para la autenticación, administración y gestión de archivos estáticos. La solución deberá ser escalable y adaptable a futuros requerimientos, facilitando tanto el desarrollo como la colaboración en equipo.

Requerimientos 🤝

- **Acceso a datos y configuración de Base de Datos:**

  Configurar el proyecto para conectarse a una base de datos relacional (SQLite para desarrollo y PostgreSQL para producción) mediante la definición de parámetros en *settings.py* y la instalación de adaptadores (como psycopg2).

- **Modelo de Datos y ORM:**

  Definir modelos representativos (por ejemplo, `Cliente`, `Cuenta`, `Transaccion`) utilizando el ORM de Django, con campos adecuados,

validaciones y relaciones (Muchos a Uno, Muchos a Muchos y Uno a Uno) para reflejar escenarios del mundo real.

- **Migraciones:**

  Utilizar comandos `makemigrations` y `migrate` para asegurar que los cambios en los modelos se apliquen correctamente al esquema de la base de datos y mantener un historial versionado.

- **Consultas Personalizadas:**

  Implementar consultas avanzadas que combinen filtros, anotaciones y, de ser necesario, ejecución de SQL personalizado (con `raw()` o `cursores`) para recuperar y manipular datos de forma eficiente y segura.

- **Operaciones CRUD:**

  Desarrollar vistas basadas en clases que permitan crear, leer, actualizar y eliminar registros, integrando formularios protegidos con tokens CSRF y configurando rutas dinámicas para identificar registros.

- **Aplicaciones Preinstaladas:**

  Aprovechar las funcionalidades de aplicaciones como *django.contrib.admin* para la administración, *django.contrib.auth* para la autenticación y gestión de usuarios, y *django.contrib.staticfiles* para la administración de recursos estáticos.

¿Qué vamos a validar? 🔍

Se validará la correcta integración y funcionamiento de los siguientes aspectos:

→ Configuración y conexión a la base de datos con Visual Studio Code y librerías gratuitas.

→ Definición y manipulación de modelos mediante el ORM, incluyendo relaciones y migraciones.

→ Ejecución de consultas personalizadas y operaciones CRUD que demuestren la robustez y eficiencia del acceso a datos.

→ Uso adecuado de las aplicaciones preinstaladas para la administración, autenticación y gestión de archivos, evidenciando su contribución al desarrollo ágil y seguro del proyecto.

→ Legibilidad, calidad del código y cumplimiento de las mejores prácticas en desarrollo.

Referencias 🦺

- Django Project – Documentación oficial
- Simple is Better than Complex – Guías y ejemplos prácticos

Paso a paso 👣

Este proyecto será avanzado paso a paso en cada clase y podrás completarlo progresivamente a través de los espacios asincrónicos en la herramienta de Visual Studio Code. También tendrás espacios de consulta en las clases sincrónicas para despejar tus dudas.

🔹 Lección 1: Configuración inicial del proyecto Django

📌 **Objetivo:** Configurar el entorno de desarrollo e iniciar el proyecto base en Django, definiendo la conexión a la base de datos.
📍 **Tareas a desarrollar:**

- Crear entorno virtual y activar en Visual Studio Code.
- Instalar Django con `pip install django`.
- Crear el proyecto base con `django-admin startproject`.
- Configurar `settings.py` para la conexión con SQLite (desarrollo) y PostgreSQL (producción).
  📂 **Archivos clave:**
- `settings.py`
- `manage.py`
- Carpeta del proyecto raíz

🔹 Lección 2: Creación de la aplicación y definición de modelos

📌 **Objetivo:** Crear la aplicación principal del proyecto e implementar el modelo de datos con el ORM de Django.
📍 **Tareas a desarrollar:**

- Crear una nueva app con `python manage.py startapp gestion`.
- Registrar la app en `INSTALLED_APPS`.
- Definir modelos como `Cliente`, `Cuenta`, `Transaccion` con sus relaciones.
- Aplicar validaciones y configuraciones en los campos del modelo.
  📂 **Archivos clave:**
- `gestion/models.py`
- `__init__.py`

🔹 Lección 3: Migraciones y sincronización de la base de datos

📌 **Objetivo:** Aplicar migraciones para reflejar los modelos en el esquema de la base de datos de forma controlada.
📍 **Tareas a desarrollar:**

- Generar migraciones con `python manage.py makemigrations`.
- Ejecutar las migraciones con `python manage.py migrate`.
- Verificar la estructura de tablas creada en SQLite.
  📂 **Comandos clave:**
- `makemigrations`, `migrate`
- Carpeta `migrations/`

🔹 Lección 4: Consultas personalizadas y manipulación de datos

📌 **Objetivo:** Implementar operaciones de acceso y manipulación de datos utilizando el ORM de Django y consultas SQL personalizadas.
📍 **Tareas a desarrollar:**

- Crear, leer, actualizar y eliminar registros desde la shell de Django.
- Usar filtros avanzados (`filter()`, `exclude()`, `annotate()`).
- Ejecutar consultas SQL con `raw()` o cursores.
  📂 **Archivos y herramientas:**
- `gestion/models.py`
- Shell interactiva (`python manage.py shell`)

🔹 Lección 5: Implementación de vistas CRUD

📌 **Objetivo:** Desarrollar vistas que permitan realizar operaciones CRUD completas sobre los modelos definidos.
📍 **Tareas a desarrollar:**

- Crear vistas basadas en clases o funciones (`CreateView`, `ListView`, etc.).
- Configurar rutas dinámicas en `urls.py`.
- Proteger formularios con tokens CSRF.
- Crear templates HTML para formularios y listados.
  📂 **Archivos clave:**
- `gestion/views.py`
- `gestion/urls.py`
- `templates/*.html`

🔹 Lección 6: Uso de aplicaciones preinstaladas en Django

📌 **Objetivo:** Integrar las aplicaciones preinstaladas de Django para facilitar la administración, autenticación y gestión de recursos.
📍 **Tareas a desarrollar:**

- Registrar modelos en el admin (`admin.py`).
- Crear superusuario para acceso a `/admin`.
- Configurar `django.contrib.auth` para login/logout.
- Usar `django.contrib.staticfiles` para servir CSS e imágenes.
  📂 **Archivos clave:**
- `admin.py`, `urls.py`, `settings.py`
- Templates: `registration/login.html`, `base.html`

🔹 Lección 7: Versionado del proyecto con Git y GitHub

📌 **Objetivo:** Gestionar el código fuente con control de versiones, promoviendo buenas prácticas colaborativas.
📍 **Tareas a desarrollar:**

- Inicializar el repositorio con `git init`.
- Crear archivo `.gitignore`.
- Hacer commits frecuentes y descriptivos.
- Subir el repositorio a GitHub y crear ramas para funcionalidades.
  📂 **Estructura en GitHub:**
- Rama `main`: código estable
- Rama `feature/modelos`: definición de modelos
- Rama `feature/crud`: vistas y formularios

Recursos 🎁

- Visual Studio Code (última versión)
- Python 3.x y Django 3.x/4.x (versión gratuita)
- SQLite para desarrollo o una instancia gratuita de PostgreSQL
- Tutoriales y blogs técnicos en línea para profundizar en cada componente

Entregables ✅

➜ **Código Fuente:** Proyecto completo en GitHub, incluyendo todos los archivos y configuraciones (settings.py, models.py, views.py, templates, etc.).

➜ **Documentación Técnica:** Informe o README.md que describa la arquitectura del proyecto, las configuraciones realizadas y las instrucciones para ejecutar la aplicación localmente.

➜ **Demostración Funcional:** Prototipo o video demostrativo que evidencie las operaciones CRUD, consultas personalizadas y el uso de aplicaciones preinstaladas.

➜ **Informe de Pruebas:** Resultados de pruebas realizadas (unitarias o de integración) que validen la integridad y funcionamiento del sistema.

Portafolio 💼

Este proyecto se integrará en tu portafolio profesional, destacando la implementación de un sistema robusto y escalable utilizando Django. Se recomienda incluir capturas de pantalla, documentación detallada y un enlace a la demo o repositorio en GitHub, resaltando los aspectos más innovadores y las buenas prácticas adoptadas durante el desarrollo.

Este proyecto "Alke Wallet" reúne los conceptos aprendidos en los siete manuales, permitiéndote aplicar de forma práctica la integración de bases de datos, el uso del ORM, la gestión de relaciones, migraciones, consultas personalizadas, operaciones CRUD y el aprovechamiento de las aplicaciones preinstaladas en Django. ¡Manos a la obra!

Resolución Paso a Paso para "Alke Wallet"

A continuación se detalla la resolución del proyecto "Alke Wallet", donde se integran todos los conceptos aprendidos en el módulo de Desarrollo Web con Django, aplicados en Visual Studio Code utilizando librerías gratuitas:

Paso 1: Configuración Inicial del Proyecto

1. Abre Visual Studio Code y abre una terminal integrada.

Crea el proyecto Django ejecutando:

```
django-admin startproject websolutions_platform
cd websolutions_platform
```

2. Configura la conexión a la base de datos en *settings.py*:
   - Para desarrollo, se usará SQLite (por defecto).
   - Agrega una nota o comentario indicando cómo se configuraría para PostgreSQL (por ejemplo, usando psycopg2 y estableciendo parámetros como `NAME`, `USER`, `PASSWORD`, `HOST` y `PORT`).

Paso 2: Creación de la Aplicación y Definición de Modelos

Crea una nueva aplicación llamada `gestion` (o el nombre que prefieras para gestionar datos) ejecutando:

```
python manage.py startapp gestion
```

1. Agrega la aplicación `gestion` a la lista `INSTALLED_APPS` en *settings.py*.

Define en *gestion/models.py* los modelos básicos necesarios. Por ejemplo, crea el modelo `Cliente`:

```
from django.db import models

class Cliente(models.Model):
```

```
    nombre = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
       telefono  =  models.CharField(max_length=20,  blank=True,
null=True)

    def __str__(self):
        return self.nombre
```

2. (Opcional) Si el proyecto requiere más entidades, define otros modelos como `Cuenta` y `Transaccion` y establece las relaciones entre ellos usando ForeignKey, ManyToManyField o OneToOneField según corresponda.

Paso 3: Generación y Aplicación de Migraciones

En la terminal, ejecuta:

```
python manage.py makemigrations gestion
```

1. Esto generará un archivo de migración (por ejemplo, *0001_initial.py*) que describe la creación de la tabla para `Cliente` (y otros modelos si los definiste).

Aplica las migraciones con:

```
python manage.py migrate
```

2.
3. Verifica que las tablas se hayan creado correctamente utilizando la shell de Django o una herramienta de administración de bases de datos.

Paso 4: Implementación de Operaciones CRUD y Consultas Personalizadas

1. **Operaciones CRUD con el ORM:**

Abre la shell de Django con:

```
python manage.py shell
```

- Realiza operaciones básicas:

Crear registros:

```
from gestion.models import Cliente
Cliente.objects.create(nombre="Ana García",
email="ana@example.com", telefono="123456789")
Cliente.objects.create(nombre="Luis Pérez",
email="luis@example.com")
```

Leer registros:

```
todos_clientes = Cliente.objects.all()
print(list(todos_clientes))
# Filtrar clientes cuyo nombre empieza con 'A'
clientes_A = Cliente.objects.filter(nombre__istartswith='A')
print(list(clientes_A))
```

Actualizar un registro:

```
cliente = Cliente.objects.get(email="luis@example.com")
cliente.telefono = "987654321"
cliente.save()
```

Eliminar un registro:

```
Cliente.objects.get(email="ana@example.com").delete()
```

2. **Consultas Personalizadas:**

Utiliza el método *raw()* para ejecutar una consulta SQL personalizada:

```
consulta = "SELECT * FROM gestion_cliente WHERE telefono IS
NOT NULL"
clientes_con_telefono = Cliente.objects.raw(consulta)
for cliente in clientes_con_telefono:
    print(cliente.nombre, cliente.email)
```

-

(Opcional) Usa conexiones y cursores para una consulta directa:

```
from django.db import connection
with connection.cursor() as cursor:
    cursor.execute("SELECT COUNT(*) FROM gestion_cliente")
    resultado = cursor.fetchone()
    print("Total de clientes:", resultado[0])
```

-

Paso 5: Integración de Aplicaciones Preinstaladas y Seguridad

1. **Administración:**

Registra el modelo `Cliente` en *gestion/admin.py* para gestionarlo a través del sitio de administración:

```
from django.contrib import admin
from .models import Cliente

class ClienteAdmin(admin.ModelAdmin):
    list_display = ('nombre', 'email', 'telefono')
    search_fields = ('nombre', 'email')

admin.site.register(Cliente, ClienteAdmin)
```

-

Ejecuta el servidor de desarrollo:

```
python manage.py runserver
```

-
- Accede a `/admin` para verificar que el modelo `Cliente` se muestra correctamente.

2. **Seguridad en Formularios:**

   - Si implementas formularios en vistas, asegúrate de incluir `{% csrf_token %}` en cada plantilla HTML para proteger contra ataques CSRF.

3. **Archivos Estáticos:**

   - Revisa que la configuración de archivos estáticos en *settings.py* esté activa y, si es necesario, crea un directorio `static/` para almacenar recursos como CSS o imágenes.

Paso 6: Documentación y Validación

1. Documenta cada paso en un archivo README.md, incluyendo:

   - Capturas de pantalla del código en Visual Studio Code (configuración, modelos, migraciones, resultados en shell y en la interfaz de administración).

   - Explicaciones sobre cómo se configuró la base de datos y se definieron los modelos.

   - Reflexiones sobre el uso del ORM, las migraciones y la integración de consultas personalizadas.

2. Prepara un informe final que resuma el proceso y detalle las pruebas realizadas para validar el correcto funcionamiento del sistema.

Paso 7: Incorporación al Portafolio

- Sube el proyecto completo a GitHub, asegurándote de incluir el código fuente, documentación técnica y capturas de pantalla.

- Destaca en tu portafolio los aspectos más relevantes del proyecto "Alke Wallet", tales como la integración de operaciones CRUD, la gestión de migraciones y el uso del ORM de Django, demostrando así tu capacidad para desarrollar aplicaciones web robustas y escalables.

¡Éxitos!

Nos vemos más adelante

alkemy